THE4Report

1st İlkim Oğul
CENG
METU
223767

2nd Onur Yaman
MATH
METU
2007961

I. Object Counting

A. General process

Initially, I tried following the vaguely similar exercise in lecture notes 11 that was based on edge filling. But flowers are much more complicated than coins sitting on a desk, so that didn't work.

Then we experimented with thresholding, and eventually arrived at the method of using the cv2 thresholding functions to dramatically emphasize the flowers, and then binarizing the image. After that, we used erosion to erase irrelevant pixels that got carried over, and then using dilation to unite flower petals into connected masses for cv2's connectedComponents function to count.

After some trial and error, the functions we used had a,b,c variables that corresponded to the threshold value, the size of the erosion kernel, and the iterations of the dilation kernel respectively.

B. A1 specific

There are MANY flowers here that are very close together, so I figured that the dilation iterator should be low to keep them from becoming a blob. Otherwise, the count1 function used here ended up identical to the count 3 used in A3. Though, it's hard to confirm anything since my rotten brain refused to actually count how many flowers there were in the image.

C. A2 specific

This one presented the challenge of taking a bulb along after binarizing. And any attempts to remove it also destroyed the bottom left flower. I 'solved' the problem by deciding that a bulb counts as a flower. Though, the dilation count had to be kept high for the bottom left flower to count as a single entity. The buds witing the center of the flowers also interfered, so they ended up setting the minimum dilation kernel size. Otherwise, the function used is identical to the one used for the prior image.

D. A3 specific

This was a weird one, as the foreground flowers were way too close together in a way that erosion couldn't seperate. Instead, I noticed how after thresholding, the edges of the flowers were much brighter than the insides, so before binarizing I turned pixels above a certain brightess to 0, and proceeded with the usual operations.

Fig. 1. Big dilation count resulted in a pixelated look. Note how the bottom-left flower is barely held together.

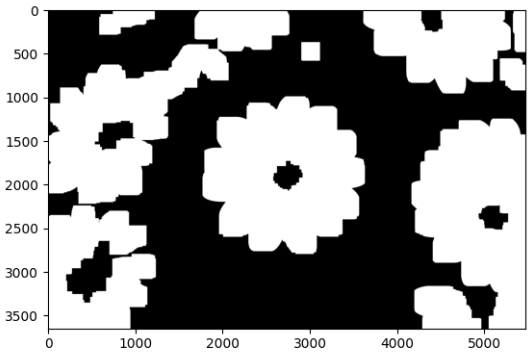
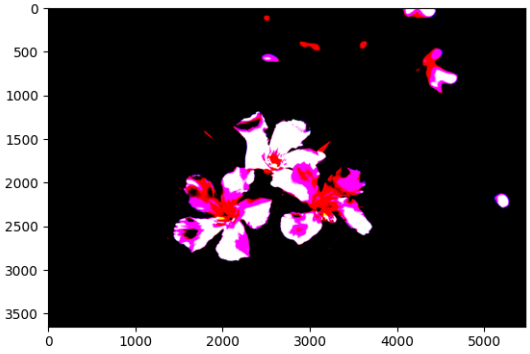


Fig. 2. The bright white parts weren't helping, had to be removed.



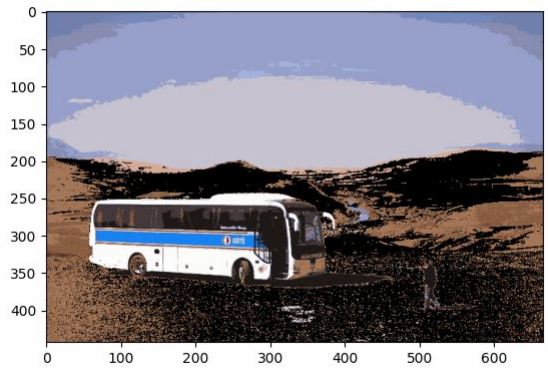
II. Segmentation

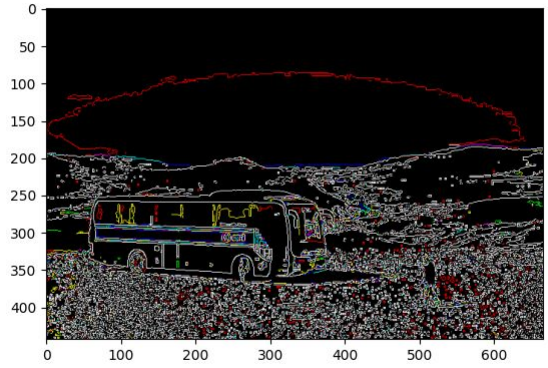
A. Segmentation Map

We used sklearn for this part. More specifically cluster.MeanShift to obtain bandwidth. Here estimatebandwidth() is the function that helps us find the number of clusters. As seen in outputs, when we change quintile from 0.05 to 0.02, the number of clusters immediately decreased from 31 to 5. This means that we will have more different colored regions. So that will clearly make it to get the corresponding adjecency

graph in in later stages.

Firstly, I decided to focus on B3 since it has the lowest size. As seen the images below





B. Boundry Overlay

C. Tree relationship structure

D. Region adjacency graph

E. Image specific problems